

Benchmark Comparatif des Bases de Données NoSQL Multi-Modèles

Étude Comparative des Performances et de l'Efficacité des Ressources

Projet de Recherche Académique

Analyse de 11 Bases de Données NoSQL
4 Familles : Document, Clé-Valeur, Colonnes, Graphe

Datasets : Amazon Reviews (568K) et Goodreads Reviews (1.85M)

Date : 15/02/2026

Table des Matières

1. Résumé Exécutif	3
2. Architecture du Système de Benchmark	4
3. Étude n°1 : Bases de Données Orientées Documents	6
3.1 MongoDB vs ArangoDB vs RavenDB	6
4. Étude n°2 : Bases de Données Clé-Valeur	10
4.1 DynamoDB vs Riak KV	10
5. Étude n°3 : Bases de Données Orientées Colonnes	14
5.1 Cassandra vs HBase	14
6. Étude n°4 : Bases de Données Orientées Graphe	18
6.1 Neo4j vs JanusGraph vs OrientDB	18
7. Synthèse Comparative Globale	22
8. Recommandations et Conclusions	24

1. Résumé Exécutif

Cette étude présente une analyse comparative approfondie de 11 bases de données NoSQL réparties en 4 familles distinctes : Document, Clé-Valeur, Colonnes et Graphe. L'objectif principal est d'évaluer les performances, l'efficacité des ressources et la scalabilité de ces systèmes dans un environnement conteneurisé Docker standardisé.

1.1 Méthodologie Générale

Le benchmark a été réalisé sur une machine locale équipée d'un processeur AMD Ryzen 7 7735HS (8 cœurs, 16 threads), 16 Go de RAM DDR5 et un SSD NVMe. Deux jeux de données réels ont été utilisés : Amazon Reviews (568K documents, ~367 Mo) et Goodreads Reviews (1.85M documents, ~1.8 Go). Chaque base de données a été testée selon un protocole standardisé incluant des phases d'insertion, de lecture et d'exportation, avec une surveillance continue des ressources via cAdvisor, Prometheus et Grafana.

1.2 Résultats Clés par Famille

Famille	Gagnant	Performance	Efficacité RAM
Document	MongoDB	3-5x plus rapide	~2 Go
Clé-Valeur	DynamoDB	5x plus rapide	~1-2.5 Go
Colonnes	HBase (Écriture) Cassandra (Lecture)	20x en écriture 2x en lecture	1-1.3 Go (HBase) 2.6-2.8 Go (Cass)
Graphe	Neo4j	4-20x plus rapide	~700-800 Mo

Conclusion principale : Les bases de données spécialisées (MongoDB pour les documents, Neo4j pour les graphes, DynamoDB pour le clé-valeur) démontrent des performances supérieures dans leur domaine respectif par rapport aux solutions multi-modèles. L'efficacité des ressources varie considérablement, avec des bases Java (JanusGraph, OrientDB, Riak KV) consommant jusqu'à 6 fois plus de RAM que leurs équivalents optimisés (Neo4j, HBase, DynamoDB).

2. Architecture du Système de Benchmark

2.1 Vue d'Ensemble de l'Architecture

Le framework de benchmarking est construit autour d'une classe de base abstraite unique (DatabaseBenchmark) qui définit le contrat pour toutes les interactions avec les bases de données. Cette approche permet à différents runners (main.py, main_kv.py, main_graph.py, main_column.py) de traiter les 11 bases de données de manière polymorphique, tout en respectant les principes SOLID de conception orientée objet.

2.2 Modèles de Données Supportés

Modèle	Bases de Données	Cas d'Usage
Document	MongoDB, ArangoDB, RavenDB	Données JSON flexibles, CMS, catalogues
Clé-Valeur	DynamoDB, Riak KV	Cache, sessions, files d'attente
Colonnes	Cassandra, HBase	Séries temporelles, logs, IoT
Graphe	Neo4j, JanusGraph, OrientDB	Réseaux sociaux, recommandations

2.3 Composants du Système

Couche d'Abstraction (src/base/benchmark_base.py) :

Définit le cycle de vie standard d'un benchmark avec les méthodes connect(), insert_data(), read_data() et measure_execution_time(). Cette classe utilise le pattern Template Method pour permettre aux sous-classes de surcharger des étapes spécifiques.

Couche d'Implémentation (src/databases/) :

Chaque base de données dispose d'un fichier d'implémentation dédié encapsulant sa logique de driver spécifique. Par exemple, MongoDB utilise pymongo, Neo4j utilise neo4j-driver, Cassandra utilise cassandra-driver, etc.

Stack de Monitoring :

L'architecture utilise une approche sidecar de monitoring via Docker Compose. cAdvisor (port 8082) collecte les métriques en temps réel (CPU, mémoire, I/O réseau) depuis les conteneurs, Prometheus (port 9090) stocke les séries temporelles, et Grafana (port 3000) fournit les visualisations.

2.4 Configuration Matérielle et Conteneurs

Composant	Spécification
-----------	---------------

Processeur	AMD Ryzen 7 7735HS
Cœurs / Threads	8 cœurs / 16 threads
Mémoire RAM	16 Go DDR5
Stockage	SSD NVMe
Système d'Exploitation	Linux (Ubuntu/Debian)

Configuration des Conteneurs Docker :

Des limites de ressources spécifiques ont été appliquées aux conteneurs gourmands en mémoire pour éviter les dépassements et assurer la stabilité des tests.

Conteneur	Limite Mémoire	Java Heap (Xmx)	Notes
DynamoDB Local	8 Go	6 Go	Augmentée pour dataset Goodreads
Riak KV	6 Go	Par défaut	Prévention des OOM kills
Cassandra	Aucune	2 Go	Configuration standard
Neo4j	Aucune	Par défaut	Pas de limite explicite
JanusGraph	Aucune	Par défaut	Pas de limite explicite
OrientDB	Aucune	Par défaut	Pas de limite explicite

2.5 Patterns de Conception Utilisés

1. Template Method : La méthode `DatabaseBenchmark.run_full_benchmark()` définit le squelette du processus de benchmark, permettant aux sous-classes de surcharger des étapes spécifiques (insert, read).

2. Strategy Pattern : Utilisation de différents scripts Runner (`main.py`, `main_graph.py`) pour sélectionner différents ensembles de stratégies de bases de données.

3. Decorator Pattern : La logique d'autorisation et de timing est encapsulée autour des méthodes principales pour séparer les préoccupations de la logique métier.

3. Étude n°1 : Bases de Données Orientées Documents

3.1 Introduction et Méthodologie

Cette étude évalue les performances de trois bases de données NoSQL orientées documents majeures : MongoDB (leader du marché), ArangoDB (solution multi-modèle) et RavenDB (base ACID transactionnelle). Le protocole de test incluait l'import en masse (Bulk Insert), des opérations CRUD complexes (Read/Update/Delete) et l'export complet des données.

3.2 Performance d'Insertion

Base de Données	Amazon (568K)	Débit	Goodreads (1.85M)	Débit
MongoDB	10.23s	55 500 docs/s	31.05s	59 500 docs/s
ArangoDB	29.32s	19 400 docs/s	144.93s	12 700 docs/s
RavenDB	41.83s	13 600 docs/s	130.39s	14 100 docs/s

Analyse : MongoDB démontre une supériorité nette avec un débit d'insertion 3 à 5 fois supérieur à ses concurrents. Sur le dataset Goodreads, MongoDB atteint 59 500 documents/seconde, soit 4.2 fois plus rapide qu'ArangoDB et 4.3 fois plus rapide que RavenDB. Cette performance s'explique par l'optimisation du format BSON et l'efficacité du moteur de stockage WiredTiger.

3.3 Performance CRUD et Export

Opération	MongoDB	ArangoDB	RavenDB	Écart Max
CRUD Amazon	1.21s	3.41s	38.07s	Mongo 30x plus rapide
CRUD Goodreads	2.56s	14.58s	75.97s	Mongo 29.7x plus rapide
Export Goodreads	29.17s	37.87s	89.08s	Mongo 3x plus rapide

Analyse : RavenDB souffre particulièrement sur les opérations CRUD, étant jusqu'à 30 fois plus lent que MongoDB. ArangoDB se positionne en seconde place avec des performances honorables mais nettement en retrait. L'export complet montre une tendance similaire, MongoDB exportant 1.85M de documents en moins de 30 secondes contre 89 secondes pour RavenDB.

3.4 Consommation des Ressources

Base de Données	CPU Moyen	RAM Moyenne	Efficacité
MongoDB	37-64%	1.1 - 2.3 Go	Haute

ArangoDB	~60%	0.4 - 2.4 Go	Moyenne
RavenDB	55-61%	2.5 - 2.8 Go	Basse

Analyse : Les trois bases consomment des niveaux de CPU similaires (55-65%), mais MongoDB termine son travail 5 fois plus rapidement, réduisant ainsi son coût CPU total. RavenDB utilise le plus de mémoire (~2.8 Go), tandis qu'ArangoDB affiche une gestion mémoire variable suggérant un garbage collection agressif. MongoDB démontre le meilleur ratio performance/ressource.

3.5 Conclusion de l'Étude Document

Gagnant : MongoDB. Il surclasse ArangoDB et RavenDB sur toutes les métriques de performance pure (vitesse d'insertion, latence CRUD, export). MongoDB est le choix par défaut pour les applications nécessitant un haut débit d'écriture et de lecture de documents.

ArangoDB se positionne comme un "bon deuxième" acceptable, particulièrement si des besoins graphes sont anticipés grâce à sa nature multi-modèle.

RavenDB se montre le moins performant dans ce benchmark spécifique, bien qu'il puisse avoir d'autres atouts (transactions ACID, intégration .NET) non évalués ici.

4. Étude n°2 : Bases de Données Clé-Valeur

4.1 Introduction et Contexte

Cette étude compare DynamoDB Local (version locale du service AWS) et Riak KV (magasin distribué basé sur Erlang VM). Les bases clé-valeur sont optimisées pour des accès rapides par clé unique, typiquement utilisées pour le caching, la gestion de sessions et les files d'attente distribuées.

4.2 Performance d'Insertion

Base de Données	Amazon (568K)	Débit (ops/s)	Goodreads (1.85M)	Débit (ops/s)
DynamoDB	118.5s	4 791	400.2s	4 620
Riak KV	598.6s	950	1846.1s	1 002
Ratio	5.0x	-	4.6x	-

Analyse : DynamoDB démontre un débit d'écriture significativement plus élevé, environ 5 fois supérieur à Riak KV. DynamoDB Local atteint 4 620 ops/s sur Goodreads, contre seulement 1 002 ops/s pour Riak. Cette différence s'explique par le coût de l'API HTTP de Riak et sa nature distribuée (même en nœud unique), comparé à l'implémentation légère de DynamoDB Local optimisée pour les tests.

4.3 Performance de Lecture

Base de Données	Amazon (1000 lectures)	Goodreads (1000 lectures)	Observation
DynamoDB	1.33s	2.32s	Très rapide
Riak KV	1.66s	1.68s	Légèrement plus rapide sur Goodreads

Analyse : Les performances de lecture sont comparables entre les deux systèmes, avec des latences négligeables (millisecondes par requête). Les deux bases excellent dans leur cas d'usage principal : la récupération clé-valeur. Les différences observées sont insignifiantes pour la plupart des applications.

4.4 Consommation des Ressources

Base de Données	CPU Moyen	RAM Moyenne	Observation
DynamoDB	66-76%	855 - 2587 Mo	Efficace, usage modéré
Riak KV	929-973%	836 - 2144 Mo	Sature le CPU (10+ cœurs)

Analyse critique : Riak KV est extrêmement gourmand en CPU, utilisant presque toute la puissance de calcul disponible (929-973%, soit 10+ cœurs) pour atteindre seulement 1/5ème du débit de DynamoDB. Cette consommation excessive suggère un coût élevé par opération dans l'architecture Erlang VM/Riak pour cette charge de travail spécifique. DynamoDB reste bien plus efficace avec 66-76% de CPU moyen.

4.5 Difficultés Techniques Rencontrées

1. Crash de Riak sur les Gros Volumes : Riak crashait systématiquement (Out-of-Memory ou timeout) lors de l'insertion du dataset Goodreads. Solution : implémentation d'une limite de mémoire stricte (6 Go) et passage au streaming des données (lecture ligne par ligne).

2. Goulot d'Étranglement HTTP : L'approche séquentielle initiale limitait Riak à ~400 ops/s. Solution : réécriture avec ThreadPoolExecutor (12 workers) et connexions persistantes, doublant le débit.

3. Erreurs de Connexion : Riak réinitialisait les connexions sous forte charge. Solution : stratégie de réessai automatique avec backoff exponentiel.

4.6 Conclusion de l'Étude Clé-Valeur

Gagnant : DynamoDB Local. Il surpasse Riak KV sur toutes les métriques intensives en écriture avec un débit ~5x supérieur et une consommation CPU ~12x inférieure. Pour un environnement de développement local ou des besoins haute performance sur un nœud unique, DynamoDB Local est le choix évident.

Les avantages de Riak en matière de tolérance aux pannes distribuée et d'architecture en anneau ne deviennent apparents que dans un scénario de cluster multi-nœuds, ce qui dépassait le cadre de cette étude.

5. Étude n°3 : Bases de Données Orientées Colonnes

5.1 Introduction et Modélisation

Cette étude évalue Apache Cassandra et Apache HBase, deux systèmes de gestion de bases de données orientées colonnes ("Wide Column Stores") conçus pour gérer de très grands volumes de données distribuées. La modélisation des données est cruciale dans ces systèmes car les requêtes doivent être connues à l'avance.

Cassandra : Utilise une clé de partition (`user_id`) pour la distribution des données et une clé de clustering (`timestamp`) pour le tri sur disque. Structure : PRIMARY KEY (`user_id`, `timestamp`) WITH CLUSTERING ORDER BY (`timestamp` DESC).

HBase : Stocke les données sous forme de chaînes d'octets triées par Row Key. Row Key = `user_id` + (`Long.MAX_VALUE` - `timestamp`) + `review_id`. L'inversion du timestamp permet de trier les avis les plus récents en premier (tri lexicographique).

5.2 Performance d'Insertion

Base de Données	Amazon (568K)	Goodreads (1.85M)	Rapport
Cassandra	1924.8s (32 min)	5410.3s (90 min)	Très lent
HBase	96.3s	331.8s (~5.5 min)	Excellent
Ratio	20x	16x	HBase domine

Analyse : HBase domine largement l'insertion massive dans cette configuration Docker single-node, avec un débit ~20x supérieur à Cassandra. L'écriture séquentielle via Thrift s'avère très efficace. Cassandra a souffert, probablement à cause de l'utilisation de BatchStatement mal tuné (anti-pattern pour gros volumes avec partitions aléatoires). Le nœud coordinateur était surchargé. Une approche asynchrone (`execute_async`) sans batch aurait été préférable.

5.3 Performance de Lecture

Base de Données	Amazon	Goodreads	Observation
Cassandra	2.94s	3.90s	Scale mieux ($O(1)$ par partition)
HBase	2.80s	7.24s	Ralentissement sur gros volumes

Analyse : Sur le petit dataset (Amazon), les performances sont identiques. Sur Goodreads, Cassandra scale mieux (3.9s vs 7.2s, soit presque 2x plus rapide). Sa structure de partitionnement par `user_id` permet un accès direct constant $O(1)$, alors que HBase doit scanner les Blocks HFile même avec la bonne RowKey, ce qui est plus coûteux à grande échelle sans optimisation (Bloom Filters).

5.4 Consommation des Ressources

Base de Données	CPU Moyen	RAM Moyenne	Observation
Cassandra	~30%	2.6 - 2.8 Go	Goulot non-CPU (I/O, lock)
HBase	>100% (pics)	1.1 - 1.3 Go	Utilisation efficace, 2x moins RAM

Analyse : HBase affiche des pics CPU très élevés (>100% sur plusieurs cœurs) pendant l'insertion rapide, montrant une utilisation efficace des ressources. Cassandra utilise un CPU plus modéré (~30%), suggérant que le goulot d'étranglement résidait dans les attentes d'I/O ou de verrous (nœud coordinateur). HBase consomme 2 fois moins de RAM pour un travail d'écriture 20 fois plus rapide.

5.5 Conclusion de l'Étude Colonnes

Vainqueur Écriture : HBase. Sa simplicité d'écriture séquentielle (même via Thrift) a écrasé Cassandra dans ce scénario d'ingestion massive "brute".

Vainqueur Lecture : Cassandra. Elle offre une latence de lecture plus stable et plus faible (presque 2x plus rapide) quand le volume de données augmente.

Efficacité : HBase a consommé 2x moins de RAM pour un travail d'écriture 20x plus rapide.

Dans le contexte de ce benchmark (Python, Docker, Time-Series), HBase est le gagnant global pour l'ingestion, tandis que Cassandra reste le roi de la lecture à faible latence à l'échelle.

6. Étude n°4 : Bases de Données Orientées Graphe

6.1 Introduction et Contexte

Cette étude compare trois bases de données orientées graphe : Neo4j (leader du marché avec moteur natif), JanusGraph (solution distribuée et évolutive) et OrientDB (base multi-modèle). Les tests incluent la création de graphes (insertion de nœuds et arêtes) et des requêtes de traversée pour des recommandations de produits basées sur les avis utilisateurs.

6.2 Performance d'Insertion (Création du Graphe)

Base de Données	Amazon (568K arêtes)	Goodreads (1.85M arêtes)	Observation
Neo4j	41.9s	121.6s	Impressionnant de rapidité
JanusGraph	168.3s	2381.2s (~40 min)	Ralentissement marqué
OrientDB	812.5s (~13.5 min)	2583.1s (~43 min)	Extrêmement lent

Analyse : Neo4j s'est montré le plus rapide et le plus efficace, insérant 1.85M d'arêtes en seulement 122 secondes. Son architecture native et l'utilisation efficace de la mémoire (via l'instruction UNWIND pour les lots) lui donnent un avantage décisif.

JanusGraph offre des performances décentes sur les petits volumes, mais ralentit considérablement sur Goodreads (40 minutes), probablement à cause de la gestion des transactions ou du backend de stockage par défaut (BerkeleyJE).

OrientDB est extrêmement lent en écriture (43 minutes pour Goodreads), soit ~20x plus lent que Neo4j. Son débit d'ingestion est nettement inférieur.

6.3 Performance de Lecture (Recommandation)

Base de Données	Amazon (20 users)	Goodreads (20 users)	Observation
Neo4j	0.39s	0.22s	Quasi-instantané
JanusGraph	1.46s	0.28s	Très bon sur Goodreads
OrientDB	0.24s	0.25s	Excellentes performances

Analyse : Les performances de lecture sont excellentes pour toutes les solutions, avec des temps de réponse en millisecondes. Les moteurs de graphe sont extrêmement performants pour les requêtes de traversée. Neo4j et JanusGraph offrent des temps de réponse quasi-instantanés sur Goodreads. OrientDB, malgré sa lenteur en écriture, offre d'excellentes performances de lecture (0.25s), rivalisant avec Neo4j sur les requêtes de traversée.

6.4 Consommation des Ressources

Base de Données	CPU Moyen	RAM Moyenne	Observation
Neo4j	146%	~700 Mo	Efficace, bon usage CPU
JanusGraph	123%	~4.2 Go	Très gourmand en RAM (Java)
OrientDB	105%	~3.5 Go	Gourmand en RAM également

Analyse : Neo4j est remarquablement économe en RAM (~700-800 Mo), soit 5 à 6 fois moins que ses concurrents basés sur Java (JanusGraph et OrientDB qui consomment respectivement 4.2 Go et 3.5 Go). Sur Goodreads, JanusGraph a maintenu une consommation RAM élevée (~5 Go). Neo4j combine efficacité mémoire et vitesse d'exécution.

6.5 Difficultés Rencontrées : Supernœud

Problème d'Incohérence du Schéma : Un défi majeur a été identifié avec le dataset Goodreads. Le code attendait un champ `product_id`, mais le JSON utilisait `book_id`. Conséquence : tous les livres ont été insérés sous un seul nœud "Product" avec l'ID "unknown", créant un "supernœud" avec 1.8 millions d'arêtes entrantes.

Impact : Les bases de données (particulièrement JanusGraph et Neo4j) se bloquaient ou crashaient en tentant de traverser/indexer ce nœud massif.

Solution : Modification des scripts d'importation pour gérer le fallback : `product_id = doc.get('product_id', doc.get('book_id', 'unknown'))`.

6.6 Conclusion de l'Étude Graphe

Gagnant incontesté : Neo4j.

Vitesse : ~4x plus rapide que JanusGraph et ~20x plus rapide qu'OrientDB en insertion.

Efficacité : Consomme 5 à 6 fois moins de RAM que ses concurrents basés sur Java (JanusGraph/OrientDB).

Stabilité : A géré les datasets Amazon (moyen) et Goodreads (grand) sans configuration complexe.

Recommandation : Pour des cas d'usage nécessitant une ingestion rapide et une faible empreinte mémoire avec des traversées de graphe complexes, Neo4j est le choix recommandé.

7. Synthèse Comparative Globale

7.1 Tableau Comparatif Multi-Familles

Famille	Base de Données	Insertion (Goodreads)	Lecture	RAM Moy.	Efficacité
Document	MongoDB	31.05s	2.56s	1-2.3 Go	★★★★★
	ArangoDB	144.93s	14.58s	0.4-2.4 Go	★★★███
	RavenDB	130.39s	75.97s	2.5-2.8 Go	★★████
Clé-Valeur	DynamoDB	400.2s	2.32s	0.9-2.6 Go	★★★★★
	Riak KV	1846.1s	1.68s	0.8-2.1 Go	★★████
Colonnes	Cassandra	5410.3s	3.90s	2.6-2.8 Go	★★████
	HBase	331.8s	7.24s	1.1-1.3 Go	★★★★█
Graphe	Neo4j	121.6s	0.22s	0.7-0.8 Go	★★★★★
	JanusGraph	2381.2s	0.28s	4.2-5.0 Go	★★████
	OrientDB	2583.1s	0.25s	3.5 Go	★★████

7.2 Observations Transversales

1. Spécialisation vs Multi-Modèle : Les bases de données spécialisées (MongoDB pour documents, Neo4j pour graphes, DynamoDB pour clé-valeur) surpassent systématiquement les solutions multi-modèles dans leur domaine respectif. L'approche "best-of-breed" l'emporte sur la polyvalence.

2. Impact de la JVM : Les bases basées sur Java (JanusGraph, OrientDB, Riak KV) consomment significativement plus de RAM (3-5 Go) que leurs équivalents natifs ou C/C++ (Neo4j, MongoDB, HBase : 0.7-2 Go). Le garbage collection et l'overhead de la JVM sont clairement mesurables.

3. Trade-offs Lecture/Écriture : Certaines bases excellent en lecture mais souffrent en écriture (OrientDB, Cassandra), d'autres sont optimisées pour l'ingestion (HBase, MongoDB). Le choix doit être guidé par le profil de charge de travail.

4. Scalabilité Verticale : Dans cet environnement single-node avec ressources limitées (16 Go RAM), l'efficacité mémoire devient critique. Neo4j et MongoDB démontrent qu'une architecture bien conçue peut faire plus avec moins.

7.3 Matrice de Décision par Cas d'Usage

Cas d'Usage	Base Recommandée	Alternative	À Éviter
CMS / Catalogue produits	MongoDB	ArangoDB	RavenDB
Cache / Sessions	DynamoDB	Redis (non testé)	Riak KV
Séries temporelles / Logs	HBase	InfluxDB (non testé)	Cassandra
Réseaux sociaux / Graphes	Neo4j	JanusGraph (distribué)	OrientDB
Analytics temps réel	MongoDB	Cassandra (tuné)	RavenDB
Recommandations	Neo4j	MongoDB + algo	OrientDB

8. Recommandations et Conclusions

8.1 Recommandations Stratégiques

Pour les Projets Nouveaux :

- **Documents flexibles (JSON)** → MongoDB (performance prouvée)
- **Relations complexes / Graphes** → Neo4j (efficacité exceptionnelle)
- **Cache distribué / Sessions** → DynamoDB ou Redis
- **Big Data / Séries temporelles** → HBase avec Hadoop ecosystem

Pour les Migrations :

- Évaluer le profil de charge (lecture/écriture, volume, latence)
- Considérer l'expertise de l'équipe (Java vs Python vs C++)
- Tester en environnement réaliste avant décision finale
- Ne pas sous-estimer le coût de la RAM pour les solutions Java

8.2 Limitations de l'Étude

Cette étude présente plusieurs limitations importantes :

1. Environnement Single-Node : Les tests ont été réalisés sur un seul nœud Docker. Les bases distribuées (Cassandra, Riak, JanusGraph) sont conçues pour briller en cluster multi-nœuds. Leurs avantages en haute disponibilité et tolérance aux pannes n'ont pas pu être évalués.

2. Tuning Minimal : Les configurations par défaut ont été utilisées. Un tuning approprié (taille de heap Java, batch sizes, thread pools) améliorerait significativement les performances de certaines bases (notamment Cassandra et Riak).

3. Cas d'Usage Spécifique : Les tests se concentrent sur l'ingestion massive et les lectures simples. D'autres patterns (transactions ACID, requêtes analytiques complexes, joins multi-tables) auraient pu favoriser différentes solutions.

4. Datasets Spécifiques : Amazon et Goodreads Reviews représentent un type particulier de données. Des workloads différents (IoT, logs serveur, données géospatiales) pourraient produire des résultats différents.

8.3 Travaux Futurs

Plusieurs pistes d'approfondissement méritent d'être explorées :

1. Benchmarks Distribués : Évaluer Cassandra, Riak et JanusGraph en configuration cluster (3-5 nœuds) pour mesurer la scalabilité horizontale et la résilience.

2. Workloads Mixtes : Tester des scénarios 80/20 lecture-écriture, des patterns OLAP (Online

Analytical Processing), et des transactions complexes.

3. Comparaisons Supplémentaires : Inclure Redis (clé-valeur), InfluxDB (séries temporelles), Elasticsearch (recherche full-text), et Dgraph (graphe distribué).

4. Cloud-Native : Comparer les versions managées (AWS DynamoDB, Azure Cosmos DB, MongoDB Atlas, Neo4j Aura) en conditions de production réelles.

5. Impact du Tuning : Étude systématique de l'impact de l'optimisation (indexation, sharding, compression, caching) sur chaque base.

8.4 Conclusion Générale

Ce benchmark comparatif a permis d'évaluer 11 bases de données NoSQL réparties en 4 familles distinctes dans des conditions standardisées. Les résultats démontrent clairement que **la spécialisation l'emporte sur la polyvalence** : MongoDB domine les documents, Neo4j les graphes, DynamoDB le clé-valeur, et HBase l'ingestion massive orientée colonnes.

Messages clés :

- **L'efficacité mémoire compte :** Dans un monde cloud où la RAM est coûteuse, Neo4j (700 Mo) vs JanusGraph (4.2 Go) représente une économie de 6x.
- **Le contexte est roi :** Aucune base n'est universellement supérieure. Le choix doit être guidé par le profil de charge de travail spécifique.
- **Le tuning est essentiel :** Les configurations par défaut ne sont jamais optimales. Les résultats peuvent être améliorés de 2-10x avec un tuning approprié.
- **La maturité technologique se voit :** MongoDB et Neo4j, leaders de leurs catégories respectives, démontrent une supériorité claire en termes de performance, stabilité et efficacité des ressources.

En conclusion, ce benchmark fournit une base factuelle pour guider les décisions architecturales, tout en soulignant l'importance de tests complémentaires adaptés aux contraintes spécifiques de chaque projet.

VERDICT FINAL

Documents : MongoDB ★★★★★
Clé-Valeur : DynamoDB ★★★★★
Colonnes (Écriture) : HBase ★★★★★■
Colonnes (Lecture) : Cassandra ★★★★★■
Graphe : Neo4j ★★★★★

*Choisissez la spécialisation pour la performance,
la polyvalence pour la flexibilité.*